

Paradigmi di Programmazione - A.A. 2021-22

Simulazione d'Esame

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **call-by-value** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x.x)((\lambda x.\lambda y.x)(xy))$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y z =  
  match (x y) with  
  | [] -> (z y)  
  | _::_ -> (z y) + 1;;
```

Esercizio 3 [Punti 7]

Si consideri il seguente tipo di dato che descrive alberi n -ari con nodi associati a valori interi, dove la `ntree list` si assume essere non vuota:

```
type ntree = Node of int * ntree list  
           | Leaf of int;;
```

Definire, usando i costrutti di programmazione funzionale in OCaml, una funzione `compact` con tipo

```
compact : ntree -> ntree
```

tale che `compact t` restituisca un albero `t'` ottenuto sostituendo ogni nodo `Node (n, [Leaf n1; ...; Leaf nk])` (ossia che non ha figli di tipo `Node`) contenuto in `t`, con un nodo `Node (n, [Leaf (n1+...+nk)])`. Esempi:

```
compact (Node(10,[Leaf 2; Leaf 3; Leaf 4])) =  
  Node (10,[Leaf 9])  
compact (Node(10,[Leaf 2; Node (3, [Leaf 5; Leaf 3]); Leaf 4])) =  
  Node (10,[Leaf 2; Node (3, [Leaf 8]); Leaf 4])  
compact (Node(10,[])) =  
  Node (10,[Leaf 0])
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml con una nuova tipologia di funzione non anonima e non ricorsiva, con la caratteristica di prevedere un valore di default per il proprio (unico) parametro formale. Si consideri il seguente esempio di dichiarazione di funzione in sintassi concreta:

```
function myFun (a=true) =  
  if a then 5 else 10  
in ... (* espressione che puo' contenere invocazioni a myFun *) ...
```

Le invocazioni:

- myFun(true)
- myFun()

restituiscono il medesimo risultato, ovvero il valore 5, mentre l'invocazione di

- myFun(false)

restituisce 10.

Si estenda l'implementazione dell'interprete MiniCaml quanto necessario per gestire la dichiarazione e l'invocazione di funzioni della nuova tipologia.